

Chapter 3

WHILE, DO-WHILE, Enhanced FOR Loops & Break/Continue

1. WHILE Loop

Used when you don't know in advance how many times to repeat — it keeps running as long as its condition is true. The condition is checked before each iteration, so if it's false from the start, the loop body never runs.

Syntax

```
while (condition) {  
    // statements  
}
```

Example 1

```
public class WhileLoopExample {  
    public static void main(String[] args) {  
        int i = 1;  
        while (i <= 5) {  
            System.out.println("Number: " + i);  
            i++;  
        }  
    }  
}
```

Number: 1
Number: 2
Number: 3
Number: 4
Number: 5

Example 2

```
public class WhileExample1 {  
    public static void main(String[] args) {  
        int i = 1;  
        while (i <= 10) {  
            System.out.println(i);  
            i++;  
        }  
    }  
}
```

Output: prints 1 through 10, each on its own line.

2. DO-WHILE Loop

Used when you want the code to run at least once, even if the condition turns out to be false — because the condition is checked after the loop body runs, not before.

Syntax

```
do {  
    // statements  
} while (condition);
```

Example

```
public class DoWhileExample {  
    public static void main(String[] args) {  
        int i = 1;  
        do {  
            System.out.println("Number: " + i);  
            i++;  
        } while (i <= 5);  
    }  
}
```

Number: 1
Number: 2
Number: 3
Number: 4
Number: 5

WHILE vs DO-WHILE — Key Difference

Feature	while	do-while
Condition checked	Before the loop body runs	After the loop body runs
Minimum executions	0 (may never run)	1 (always runs at least once)
Typical use	When you're not sure the block should run at all	When the block must run at least once (e.g., menus, input prompts)

3. Enhanced FOR Loop (For-Each Loop)

Used to iterate through arrays or collections easily, without needing an index variable or manually checking bounds.

Syntax

```
for (dataType variable : arrayName) {  
    // statements  
}
```

Example — Basic Iteration

```

public class EnhancedForExample {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};
        for (int num : numbers) {
            System.out.println(num);
        }
    }
}
10
20
30
40
50

```

Example 1 — Print All Elements of an Integer Array

```

public class EnhancedFor1 {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        for (int num : arr) {
            System.out.println(num);
        }
    }
}

```

Example 2 — Sum of Elements in an Integer Array

```

public class EnhancedFor2 {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        int sum = 0;
        for (int num : arr) {
            sum += num;
        }
        System.out.println("Sum = " + sum);
    }
}

```

Output: Sum = 15

Example 3 — Find the Largest Element in an Integer Array

```

public class EnhancedFor3 {
    public static void main(String[] args) {
        int[] arr = {10, 45, 23, 67, 34};
        int max = arr[0];
        for (int num : arr) {
            if (num > max) {
                max = num;
            }
        }
    }
}

```

```

        }
    }
    System.out.println("Largest element = " + max);
}

```

Output: Largest element = 67

Note: the enhanced for loop is great for reading values, but it has no index — you cannot use it to modify array elements in place, skip elements, or go backward. Use a regular for loop when you need an index.

4. Break Statement

The break statement is used to exit a loop or switch statement immediately, even before its condition becomes false. When the program encounters break inside a loop, it jumps out of the loop and continues execution from the next statement after the loop.

Example 1 — break in a loop

```

public class BreakExample1 {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 5) {
                break; // loop stops when i = 5
            }
            System.out.println(i);
        }
    }
}
1
2
3
4

```

Explanation: the loop stops as soon as i becomes 5 — it "breaks" out of the loop entirely, so 5 through 10 are never printed.

Example 2 — break in a switch statement

```

public class BreakExample3 {
    public static void main(String[] args) {
        int day = 3;
        switch (day) {
            case 1: System.out.println("Monday"); break;
            case 2: System.out.println("Tuesday"); break;
            case 3: System.out.println("Wednesday"); break;
            default: System.out.println("Invalid day");
        }
    }
}

```

```
    }  
}
```

Output: Wednesday. Here break prevents "fall-through" into the next case.

5. Continue Statement

The continue statement is used to skip the current iteration of a loop and move to the next iteration. When continue executes, the remaining code inside the loop for that iteration is skipped, and the loop moves on to check its condition again for the next pass.

Example 1 — continue in a for loop

```
public class ContinueExample1 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i == 5) {  
                continue; // skip iteration when i = 5  
            }  
            System.out.println(i);  
        }  
    }  
}
```

```
1  
2  
3  
4  
6  
7  
8  
9  
10
```

Explanation: when i is 5, the loop skips printing and continues with the next iteration — notice 5 is missing but the loop doesn't stop.

Example 2 — continue in a while loop

```
public class ContinueExample2 {  
    public static void main(String[] args) {  
        int i = 0;  
        while (i < 10) {  
            i++;  
            if (i % 2 == 0) {  
                continue; // skip even numbers  
            }  
            System.out.println(i);  
        }  
    }  
}
```

```
}  
1  
3  
5  
7  
9
```

Example 3 — continue in nested loops

continue only skips the current iteration of the loop it is directly written in (the innermost one) — it does not affect the outer loop.

```
public class ContinueExample3 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 3; j++) {  
                if (j == 2) continue;  
                System.out.println("i = " + i + ", j = " + j);  
            }  
        }  
    }  
}
```

```
i = 1, j = 1  
i = 1, j = 3  
i = 2, j = 1  
i = 2, j = 3  
i = 3, j = 1  
i = 3, j = 3
```

Explanation: whenever j equals 2, only that inner-loop pass is skipped — j = 1 and j = 3 still print normally for every value of i.

6. Difference Between break and continue

Feature	break	continue
Purpose	Exits the loop completely	Skips the current iteration
Used in	Loops, switch	Loops only
Control flow	Jumps outside the loop	Moves to next iteration
Example effect (i == 5)	Stops at i = 5	Skips i = 5, continues after

7. Tips for Choosing and Using Loops Correctly

1. Choose for when you know the number of repetitions in advance (e.g., "do this 5 times", "loop through this array").
2. Choose while when the number of repetitions depends on a condition that might change unpredictably (e.g., "keep asking until the user enters a valid number").
3. Choose do-while specifically when the loop body must run at least once regardless of the condition — classic use case: displaying a menu before checking if the user wants to exit.
4. Choose the enhanced for loop when you just need to read every element of an array or collection in order, and don't need the index, don't need to modify elements, and don't need to skip or go backward.
5. Always update the loop control variable inside while and do-while loops (e.g., `i++`). Forgetting this is the most common cause of an infinite loop.
6. Use `break` to stop a loop entirely once you've found what you're looking for (e.g., searching an array and stopping at the first match) — this avoids wasting time on remaining iterations.
7. Use `continue` when you want to skip specific unwanted values but still keep processing everything else in the loop (e.g., skipping even numbers, skipping invalid entries).
8. Remember `break` always needs to be inside a loop or switch — a `break` outside either is a compile error.
9. In nested loops, `break` and `continue` only affect the innermost loop they're written in — they do not automatically stop or skip the outer loop.
10. Trace through the loop by hand with a small range first (e.g., 1 to 5 instead of 1 to 1000) to confirm your `break/continue` condition behaves exactly as expected before running it on real data.